

Self-Contained Fixed-Branch Proofs for Zhao–Cui Tensor-Train Sequential Filtering

BayesFilter P12 Proof Expansion Note

2026-05-31

Abstract

This note expands the fixed-branch Zhao–Cui tensor-train derivative contract into a self-contained proof document. It first derives the ordinary Bayesian filtering recursion, then explains the Zhao–Cui squared tensor-train construction in BayesFilter notation, and finally introduces a deterministic fixed-branch variant. Two propositions are proved. Proposition 1 proves that the fixed-branch squared-TT recursion defines a normalized nonnegative approximate Bayesian filter. Proposition 2 proves that the analytical gradient is the exact derivative of the declared approximate scalar

$$\widehat{\ell}_T^{\text{TT}}(\alpha) = \sum_{t=1}^T \{\log \widehat{z}_t(\alpha) - c_t(\alpha)\}.$$

The proofs are for the declared fixed branch only. They do not prove posterior accuracy, HMC convergence, or differentiability of the adaptive TT-cross, rank-changing, sample-changing companion code.

1 Introduction And Reader Contract

The problem is sequential inference in a state-space model. At each time t , one wants a filtering density over the current state and unknown parameters after seeing data $y_{1:t}$. The exact density is usually not available in closed form. Zhao and Cui propose to approximate the high-dimensional density that appears inside the filtering recursion by a tensor-train (TT) representation of a square-root density, then square that TT and add a small defensive term so the resulting approximation is nonnegative. The squared representation also gives marginal densities and conditional Knothe–Rosenblatt (KR) maps.

This note separates three statements.

1. The Zhao–Cui construction is a source-grounded high-dimensional sequential approximation strategy. This is supported by their Algorithm 1, their squared-TT construction around their equation (13), Algorithm 2, and their discussion of the evidence normalizer in Section 4.1 [1].
2. The BayesFilter fixed-branch variant is a mathematically declared approximate filtering recursion. The proof is a project derivation: it shows that the output densities are nonnegative and normalized.
3. The fixed-branch analytical gradient differentiates the same approximate scalar that the fixed-branch recursion computes. This is also a project derivation. It requires fixed TT ranks, fixed interpolation or least-squares systems, fixed basis maps, fixed random/debug arrays, fixed rounding choices, fixed QR/SVD/Cholesky branches, and a declared rule for the stabilizing constant.

The note does not prove that the approximation equals the exact posterior filter. It proves correctness of the approximate recursion and correctness of the gradient for the declared approximate scalar. Accuracy must be studied by approximation theory or numerical experiments.

2 State-Space Filtering From First Principles

Let $x_t \in \mathcal{X}_t \subseteq \mathbb{R}^{d_x}$ be the state at time t , let $\vartheta \in \Theta \subseteq \mathbb{R}^{d_\vartheta}$ be a static parameter, and let y_t be the observed datum. A dominated state-space model specifies

$$p_0(x_0, \vartheta), \quad f_\alpha(x_t \mid x_{t-1}, \vartheta), \quad g_\alpha(y_t \mid x_t, \vartheta),$$

where α denotes an external model parameter with respect to which a future likelihood-gradient calculation may differentiate. The parameter ϑ is a random coordinate in the filtering distribution; α is the coordinate of the scalar derivative in this note.

Assume that the filter at time $t - 1$ has density

$$p_{t-1}(x_{t-1}, \vartheta \mid y_{1:t-1})$$

with respect to $dx_{t-1}d\vartheta$. The prediction density is obtained by integrating over the previous state:

$$p(x_t, \vartheta \mid y_{1:t-1}) = \int f_\alpha(x_t \mid x_{t-1}, \vartheta) p_{t-1}(x_{t-1}, \vartheta \mid y_{1:t-1}) dx_{t-1}.$$

Bayes' rule then gives the unnormalized update

$$\tilde{p}_t(x_t, \vartheta) = g_\alpha(y_t \mid x_t, \vartheta) p(x_t, \vartheta \mid y_{1:t-1}).$$

Its integral

$$Z_t(\alpha) = \int g_\alpha(y_t \mid x_t, \vartheta) f_\alpha(x_t \mid x_{t-1}, \vartheta) p_{t-1}(x_{t-1}, \vartheta \mid y_{1:t-1}) dx_{t-1} d\vartheta$$

is the one-step evidence increment $p_\alpha(y_t \mid y_{1:t-1})$. The exact filter is therefore

$$p_t(x_t, \vartheta \mid y_{1:t}) = \frac{g_\alpha(y_t \mid x_t, \vartheta) \int f_\alpha(x_t \mid x_{t-1}, \vartheta) p_{t-1}(x_{t-1}, \vartheta \mid y_{1:t-1}) dx_{t-1}}{Z_t(\alpha)}.$$

It is often useful to keep the previous state inside the object being approximated. Define

$$u_t = (x_t, \vartheta, x_{t-1})$$

and

$$q_t(u_t; \alpha) = p_{t-1}(x_{t-1}, \vartheta \mid y_{1:t-1}) f_\alpha(x_t \mid x_{t-1}, \vartheta) g_\alpha(y_t \mid x_t, \vartheta).$$

Then $Z_t(\alpha) = \int q_t(u_t; \alpha) du_t$, and the exact filter is the marginal of the normalized joint density q_t/Z_t over the old state:

$$p_t(x_t, \vartheta \mid y_{1:t}) = \int \frac{q_t(x_t, \vartheta, x_{t-1}; \alpha)}{Z_t(\alpha)} dx_{t-1}.$$

This is the mathematical reason Zhao–Cui approximate the nonseparable density over $(x_t, \vartheta, x_{t-1})$: once this joint object can be represented and integrated, filtering and evidence increments follow by normalization and marginalization.

3 Tensor Trains From First Principles

A tensor train is a separated representation of a high-dimensional function. Let $r = (r_1, \dots, r_D)$ and let $\{b_{k,a}\}_{a=1}^{n_k}$ be basis functions for the k -th coordinate. A scalar function $\phi(r)$ has a functional TT representation if

$$\phi(r) = G_1(r_1)G_2(r_2) \cdots G_D(r_D),$$

where

$$G_k(r_k) \in \mathbb{R}^{\rho_{k-1} \times \rho_k}, \quad \rho_0 = \rho_D = 1.$$

The integers ρ_k are TT ranks. Each matrix-valued core can be expanded in the one-dimensional basis:

$$G_k(r_k) = \sum_{a=1}^{n_k} C_{k,a} b_{k,a}(r_k), \quad C_{k,a} \in \mathbb{R}^{\rho_{k-1} \times \rho_k}.$$

Evaluation of $\phi(r)$ is a chain of matrix multiplications, rather than a full tensor-product sum over $n_1 \cdots n_D$ coefficients.

If $\nu = \nu_1 \otimes \cdots \otimes \nu_D$ is the reference measure used by the basis, define one-dimensional mass matrices

$$M_k[a, b] = \int b_{k,a}(r_k) b_{k,b}(r_k) d\nu_k(r_k).$$

The integral of ϕ^2 is obtained by the backward contractions

$$R_D = 1, \quad R_{k-1} = \sum_{a,b=1}^{n_k} M_k[a, b] C_{k,a} R_k C_{k,b}^\top, \quad k = D, \dots, 1.$$

Then $R_0 = \int \phi(r)^2 d\nu(r)$. The contraction is exact for the declared TT cores and mass matrices; approximation enters through the construction of the cores, not through this algebraic contraction.

4 Zhao–Cui Sequential Squared-TT Algorithm

Zhao–Cui’s Algorithm 1 constructs, at each time t , a nonseparable density over $(x_t, \vartheta, x_{t-1})$, approximates it in TT form, integrates the approximation to get a normalizer, and marginalizes to obtain the next filtering object [1]. In their notation, the core density is the analogue of

$$q_t(x_t, \vartheta, x_{t-1}) = \hat{\pi}_{t-1}(x_{t-1}, \vartheta \mid y_{1:t-1}) f(x_t \mid x_{t-1}, \vartheta) g(y_t \mid x_t, \vartheta).$$

The exact integral of q_t is the evidence increment for the exact filter if $\hat{\pi}_{t-1}$ is exact; with $\hat{\pi}_{t-1}$ approximate, it is the evidence increment for the declared approximate recursion.

The squared-TT construction approximates the square root of an unnormalized nonnegative density by a TT:

$$\sqrt{q_t(u_t)} \approx \phi_t(u_t) = G_{t,1}(u_{t,1}) \cdots G_{t,D}(u_{t,D}).$$

Instead of using a sign-changing TT density directly, Zhao–Cui square the TT and add a defensive density:

$$\hat{q}_t(u_t) = \phi_t(u_t)^2 + \tau_t \lambda_t(u_t), \quad \tau_t \geq 0, \quad \lambda_t \geq 0, \quad \int \lambda_t = 1.$$

The approximate normalizer is

$$\hat{Z}_t = \int \hat{q}_t(u_t) du_t.$$

Zhao–Cui’s Section 4.1 identifies the corresponding exact joint normalizer as evidence; their Algorithm 2 applies the squared-TT construction sequentially. The companion code follows the same scalar path: `models/full_sol.m` updates `logmarginal_likelihood` by `log(sirt.z)-const`, while `@TTSIRT/marginalise.m` sets `obj.z = obj.fun.z + obj.tau`.

The conditional KR transport is a sampling and conditioning mechanism built from marginal and conditional densities obtained by integrating the squared TT [1, 2]. For the two propositions below, the transport role is secondary: we need only the nonnegative squared-TT density, its normalizer, and the marginalization that produces the next filter. The KR maps explain how the same approximation can also generate samples, smoothing paths, and conditional parameter/state draws.

Published-style recursion. At a high level, the Zhao–Cui-style squared-TT filtering recursion is:

1. Start from a prior or previous approximate filter $\hat{p}_{t-1}(x_{t-1}, \vartheta)$.
2. Form the unnormalized joint density q_t in (4).
3. Build a TT approximation $\phi_t \approx \sqrt{q_t}$ using the selected basis, variable ordering, TT-cross/ALS/rank policy, and coordinate maps.
4. Define $\hat{q}_t = \phi_t^2 + \tau_t \lambda_t$ and compute $\hat{Z}_t = \int \hat{q}_t$.
5. Normalize and marginalize:

$$\hat{p}_t(x_t, \vartheta) = \int \hat{q}_t(x_t, \vartheta, x_{t-1}) / \hat{Z}_t \, dx_{t-1}.$$

6. Use the marginal and conditional densities implied by the squared TT to build conditional KR maps when sampling, smoothing, or parameter/state draws are needed.

The published and companion-code algorithms adapt many choices: cross or interpolation points, ranks, truncations, random/debug samples, stopping decisions, preconditioner branches, and QR/SVD signs or active subspaces. Such adaptation is natural for filtering. It is not a single globally smooth map from model parameter to likelihood scalar. That is why the next section declares a fixed branch.

5 Fixed-Branch TT Filtering Variant

Definition 1 (Fixed branch). A fixed branch consists of fixed state ordering, basis families, reference domains, interpolation or cross sets, TT ranks, local solve schedule, ALS sweep count, random/debug sample arrays, truncation decisions, QR/SVD/Cholesky signs and active ranks, defensive density λ_t , defensive mass rule τ_t , coordinate maps, and the stabilizing constant rule c_t . Along this branch, each of these choices is treated as a declared smooth object or as a fixed numerical object.

The stabilizing constant appears because the code approximates a shifted potential. If $V_{t,0}$ is an unshifted potential and

$$V_t(r; \alpha) = V_{t,0}(r; \alpha) - c_t(\alpha),$$

then the shifted normalizer $\widehat{z}_t$ corresponds to the original-scale increment $\exp(-c_t)\widehat{z}_t$. Thus the declared scalar is

$$\widehat{\ell}_T^{\text{TT}}(\alpha) = \sum_{t=1}^T \{\log \widehat{z}_t(\alpha) - c_t(\alpha)\}.$$

If a future implementation freezes c_t numerically rather than differentiating the selected branch rule, it must name a different scalar.

Fixed-branch recursion. Given $\widehat{p}_{t-1}^{\text{TT}}$, the fixed-branch step is:

1. form q_t from $\widehat{p}_{t-1}^{\text{TT}}$, f_α , and g_α ;
2. apply the fixed coordinate/basis maps and stabilizing constant;
3. solve the fixed TT-core construction problem, giving cores $G_{t,1}, \dots, G_{t,D}$;
4. define $\widehat{q}_t = \phi_t^2 + \tau_t \lambda_t$ and $\widehat{z}_t = \int \widehat{q}_t$;
5. normalize and marginalize to obtain $\widehat{p}_t^{\text{TT}}$;
6. store every branch object needed to reproduce the same scalar and its derivative.

The fixed branch is less adaptive than the research code. Its purpose is not to improve filtering accuracy. Its purpose is to define a deterministic approximate scalar and a same-scalar analytical gradient.

6 Proposition 1: Fixed-Branch TT Recursion Is A Normalized Approximate Filter

Assumption 1 (Dominated fixed-branch recursion). For every t , all state and parameter densities are dominated by fixed measures. The previous approximate filter $\widehat{p}_{t-1}^{\text{TT}}$ is nonnegative and integrates to one. The transition and observation densities f_α and g_α are nonnegative and measurable. The fixed branch produces a measurable square-root approximation ϕ_t . The defensive density satisfies $\lambda_t \geq 0$ and $\int \lambda_t = 1$. The defensive mass satisfies $\tau_t \geq 0$. Finally,

$$0 < \widehat{z}_t(\alpha) = \int \{\phi_t(u_t; \alpha)^2 + \tau_t(\alpha) \lambda_t(u_t)\} du_t < \infty.$$

Proposition 1 (Fixed-branch squared-TT recursion defines a normalized approximate filter). *Under the dominated fixed-branch recursion assumptions, define*

$$q_t(u_t; \alpha) = \widehat{p}_{t-1}^{\text{TT}}(x_{t-1}, \vartheta; \alpha) f_\alpha(x_t | x_{t-1}, \vartheta) g_\alpha(y_t | x_t, \vartheta),$$

$$\widehat{q}_t(u_t; \alpha) = \phi_t(u_t; \alpha)^2 + \tau_t(\alpha) \lambda_t(u_t), \quad \widehat{\pi}_t(u_t; \alpha) = \frac{\widehat{q}_t(u_t; \alpha)}{\widehat{z}_t(\alpha)},$$

and

$$\widehat{p}_t^{\text{TT}}(x_t, \vartheta; \alpha) = \int \widehat{\pi}_t(x_t, \vartheta, x_{t-1}; \alpha) dx_{t-1}.$$

Then $\widehat{p}_t^{\text{TT}}$ is a nonnegative normalized density. By induction from a normalized prior, the recursion defines a sequence of normalized approximate filtering densities.

Proof. The function ϕ_t^2 is nonnegative pointwise, and $\tau_t \lambda_t$ is nonnegative because $\tau_t \geq 0$ and $\lambda_t \geq 0$. Hence $\hat{q}_t \geq 0$. By assumption, $\hat{z}_t = \int \hat{q}_t$ is finite and strictly positive, so $\hat{\pi}_t = \hat{q}_t / \hat{z}_t$ is well defined and nonnegative. Moreover,

$$\int \hat{\pi}_t(u_t; \alpha) du_t = \frac{1}{\hat{z}_t(\alpha)} \int \hat{q}_t(u_t; \alpha) du_t = 1.$$

Now integrate the marginal in (1). Tonelli's theorem applies to the nonnegative integrand, so

$$\begin{aligned} \int \hat{p}_t^{\text{TT}}(x_t, \vartheta; \alpha) dx_t d\vartheta &= \int \int \hat{\pi}_t(x_t, \vartheta, x_{t-1}; \alpha) dx_{t-1} dx_t d\vartheta \\ &= \int \hat{\pi}_t(u_t; \alpha) du_t = 1. \end{aligned}$$

Thus the marginal is nonnegative and normalized. The base case is the normalized prior. If $\hat{p}_{t-1}^{\text{TT}}$ is normalized, the argument above proves that $\hat{p}_t^{\text{TT}}$ is normalized. Induction over t therefore proves the claim. $\square$

Corollary 1 (Exactness only under zero approximation error). *If, at every time t , the fixed-branch squared-TT object satisfies $\hat{q}_t(u_t; \alpha) = q_t(u_t; \alpha)$ almost everywhere and no shifted constant changes the scale, then $\hat{z}_t = Z_t$ and the recursion recovers the exact Bayesian filtering recursion. Otherwise it is a normalized approximate filtering recursion.*

Proof. If $\hat{q}_t = q_t$, then their integrals agree, so $\hat{z}_t = Z_t$. The normalized joint density $\hat{q}_t / \hat{z}_t$ equals q_t / Z_t , and marginalizing out x_{t-1} gives the exact filter by (2). If $\hat{q}_t \neq q_t$, Proposition 1 still proves normalization and nonnegativity, but it does not imply equality with the exact posterior. $\square$

7 Proposition 2: Same-Scalar Analytical Gradient

Assumption 2 (Fixed-branch differentiability). The assumptions of Proposition 1 hold. In addition, all branch choices are fixed. Along that branch, the model densities, previous approximate filter, coordinate maps, basis maps, core coefficients, defensive mass τ_t , and stabilizing constant c_t are differentiable in the external parameter coordinate α_i . Differentiation may be interchanged with the finite contractions and with the displayed integrals. Fixed interpolation or least-squares systems used to construct TT cores are nonsingular, or a specific differentiable pseudoinverse branch is declared. Each normalizer $\hat{z}_t$ is positive.

Proposition 2 (Analytical gradient differentiates the declared fixed-branch approximate likelihood). *Under the fixed-branch differentiability assumptions, the derivative of*

$$\hat{\ell}_T^{\text{TT}}(\alpha) = \sum_{t=1}^T \{\log \hat{z}_t(\alpha) - c_t(\alpha)\}$$

is

$$\partial_i \hat{\ell}_T^{\text{TT}}(\alpha) = \sum_{t=1}^T \left[\frac{\partial_i \hat{z}_t(\alpha)}{\hat{z}_t(\alpha)} - \partial_i c_t(\alpha) \right],$$

where

$$\partial_i \hat{z}_t = 2 \int \phi_t(u; \alpha) \partial_i \phi_t(u; \alpha) du + \partial_i \tau_t$$

when the defensive density λ_t is fixed and normalized. If λ_t also depends on α_i , then the extra term $\tau_t \int \partial_i \lambda_t du$ appears; it vanishes for normalized fixed-support λ_t because $\partial_i \int \lambda_t = 0$. Moreover, when the TT cores are produced by the displayed fixed interpolation or fixed weighted least-squares equations, the core sensitivities are obtained by differentiating exactly those equations, and the recursion includes the previous-filter derivative in (7).

Proof. The scalar is a finite sum of differentiable terms on the fixed branch. Since $\hat{z}_t > 0$,

$$\partial_i \{\log \hat{z}_t - c_t\} = \frac{\partial_i \hat{z}_t}{\hat{z}_t} - \partial_i c_t.$$

Summing over t proves (2). Next,

$$\hat{z}_t = \int \{\phi_t(u; \alpha)^2 + \tau_t(\alpha) \lambda_t(u)\} du.$$

By the assumed interchange of differentiation and integration,

$$\partial_i \hat{z}_t = \int 2\phi_t(u; \alpha) \partial_i \phi_t(u; \alpha) du + \partial_i \tau_t \int \lambda_t(u) du.$$

Because $\int \lambda_t = 1$, this gives (2).

It remains to derive $\partial_i \phi_t$ from the same computation path. If

$$\phi_t(u) = G_{t,1}(u_1) \cdots G_{t,D}(u_D),$$

then repeated use of the product rule gives

$$\partial_i \phi_t(u) = \sum_{k=1}^D G_{t,1}(u_1) \cdots \partial_i G_{t,k}(u_k) \cdots G_{t,D}(u_D).$$

Write each core in a fixed basis:

$$G_{t,k}(u_k) = \sum_{a=1}^{n_k} C_{t,k,a} b_{k,a}(u_k).$$

When the basis is fixed, the derivative of the core is obtained by replacing $C_{t,k,a}$ with $\partial_i C_{t,k,a}$. This is not an extra approximation: it is the coordinate derivative of the same core expansion.

Using the mass matrices M_k in (3), define

$$R_{t,D} = 1, \quad R_{t,k-1} = \sum_{a,b=1}^{n_k} M_k[a, b] C_{t,k,a} R_{t,k} C_{t,k,b}^\top.$$

Then the square part of the normalizer is $R_{t,0}$, and the differentiated contraction is

$$\begin{aligned} \partial_i R_{t,k-1} = \sum_{a,b=1}^{n_k} M_k[a, b] & \left[(\partial_i C_{t,k,a}) R_{t,k} C_{t,k,b}^\top + C_{t,k,a} (\partial_i R_{t,k}) C_{t,k,b}^\top \right. \\ & \left. + C_{t,k,a} R_{t,k} (\partial_i C_{t,k,b})^\top \right]. \end{aligned}$$

This is just the product rule applied to $C_{t,k,a} R_{t,k} C_{t,k,b}^\top$, propagated backward from $\partial_i R_{t,D} = 0$. Therefore the differentiated contraction is the derivative of the same direct mass-contraction value $R_{t,0}$ used in the declared normalizer.

Core sensitivities come from the fixed core-construction equations. For an exact fixed interpolation equation

$$A_{t,k}(\alpha)g_{t,k}(\alpha) = b_{t,k}(\alpha),$$

differentiation gives

$$A_{t,k}\partial_i g_{t,k} = \partial_i b_{t,k} - (\partial_i A_{t,k})g_{t,k}.$$

because

$$\partial_i(A_{t,k}g_{t,k}) = (\partial_i A_{t,k})g_{t,k} + A_{t,k}\partial_i g_{t,k} = \partial_i b_{t,k}.$$

For a fixed weighted least-squares update

$$g_{t,k} = \arg \min_g \|W_{t,k}^{1/2}(A_{t,k}g - b_{t,k})\|_2^2,$$

with fixed $W_{t,k}$ and full-rank $N_{t,k} = A_{t,k}^\top W_{t,k} A_{t,k}$, the normal equations are

$$A_{t,k}^\top W_{t,k} A_{t,k} g_{t,k} = A_{t,k}^\top W_{t,k} b_{t,k}.$$

Differentiating them gives

$$\begin{aligned} N_{t,k}\partial_i g_{t,k} &= (\partial_i A_{t,k})^\top W_{t,k}(b_{t,k} - A_{t,k}g_{t,k}) \\ &\quad + A_{t,k}^\top W_{t,k}(\partial_i b_{t,k} - (\partial_i A_{t,k})g_{t,k}). \end{aligned}$$

The right hand side is computable from the fixed branch once the derivative of the target square-root evaluations is known. It is also the derivative of the same least-squares update, because it is obtained by differentiating the normal equations that define that update.

The target square-root evaluations depend recursively on the previous approximate filter. From

$$q_t = \hat{p}_{t-1}^{\text{TT}} f_\alpha g_\alpha,$$

where the arguments are those in (4),

$$\partial_i \log q_t = \partial_i \log \hat{p}_{t-1}^{\text{TT}} + \partial_i \log f_\alpha + \partial_i \log g_\alpha.$$

If

$$\hat{p}_{t-1}^{\text{TT}}(v; \alpha) = a_{t-1}(v; \alpha) / \hat{z}_{t-1}(\alpha),$$

then

$$\partial_i \log \hat{p}_{t-1}^{\text{TT}}(v; \alpha) = \frac{\partial_i a_{t-1}(v; \alpha)}{a_{t-1}(v; \alpha)} - \frac{\partial_i \hat{z}_{t-1}(\alpha)}{\hat{z}_{t-1}(\alpha)}.$$

The numerator a_{t-1} is a marginal contraction of the previous squared-TT object. To see the derivative explicitly, write the previous marginal numerator as

$$a_{t-1}(v; \alpha) = \int \{\phi_{t-1}(v, w; \alpha)^2 + \tau_{t-1}(\alpha)\lambda_{t-1}(v, w)\} dw,$$

where w denotes the coordinates eliminated from the previous joint approximation. Under the same fixed-branch differentiation assumptions,

$$\partial_i a_{t-1}(v; \alpha) = 2 \int \phi_{t-1}(v, w; \alpha) \partial_i \phi_{t-1}(v, w; \alpha) dw + \partial_i \tau_{t-1}(\alpha) \int \lambda_{t-1}(v, w) dw,$$

with the corresponding $\partial_i \lambda_{t-1}$ term if the defensive density itself is parameter-dependent. In TT coordinates, (7) is evaluated by the same marginal contraction used for a_{t-1} , except that one core

at a time is replaced by its differentiated core, exactly as in (7) and (7). Thus the derivative is recursive in time in the same way that the filtering density is recursive.

Combining (2), (7), and (7), the direct mass-contraction version of the same-scalar score is

$$\partial_i \widehat{\ell}_T^{\text{TT}} = \sum_{t=1}^T \left[\frac{\partial_i R_{t,0} + \partial_i \tau_t}{R_{t,0} + \tau_t} - \partial_i c_t \right].$$

This is a full analytical gradient for the declared fixed-branch value path, provided the selected core-construction branch and all previous-filter contractions are differentiated by the displayed equations. $\square$

Corollary 2 (Adaptive code is not globally differentiated by this proof). *If TT-cross pivots, ranks, random/debug samples, truncation decisions, preconditioner branches, QR/SVD active subspaces, selected minimizers for c_t , or reapproximation triggers change with α , the formulas above remain derivatives of the frozen local branch only. They are not a global ordinary derivative of the adaptive companion code.*

Proof. The proof of Proposition 2 assumes fixed differentiable equations. A changed pivot set, rank, sample array, truncation, or selected minimizer changes the equations defining the scalar. The derivative of one branch is therefore not the derivative of a single globally smooth map across branch changes. $\square$

8 Implementation Checklist

A future BayesFilter implementation of the fixed-branch variant must specify:

Object	Required specification
Filtering model	p_0, f_α, g_α , dimensions of x_t, ϑ , and $u_t = (x_t, \vartheta, x_{t-1})$.
Approximation branch	Fixed basis, domains, ordering, ranks, interpolation or least-squares equations, local solve schedule, and all sample arrays.
Approximate density	$\phi_t^2 + \tau_t \lambda_t$, with λ_t normalized and $\widehat{z}_t > 0$.
Filtering output	Normalized joint $\widehat{\pi}_t$ and marginal $\widehat{p}_t^{\text{TT}}(x_t, \vartheta)$.
Scalar	Exactly $\sum_t \{\log \widehat{z}_t - c_t\}$, or a renamed scalar if c_t or the normalizer path changes.
Derivative	Core sensitivities, previous-filter sensitivities, $\partial_i R_{t,0}$, $\partial_i \tau_t$, and $\partial_i c_t$.
Diagnostics	Branch stability under finite differences, positive finite normalizers, rank stability, and same-scalar value-gradient parity.

9 Finite-Difference Parity Required Before HMC Use

For each parameter coordinate α_i , a future implementation should check that the value path and derivative path are identical. With branch objects frozen,

$$\frac{\widehat{\ell}_T^{\text{TT}}(\alpha + h e_i) - \widehat{\ell}_T^{\text{TT}}(\alpha - h e_i)}{2h} \approx \partial_i \widehat{\ell}_T^{\text{TT}}(\alpha).$$

If the finite-difference calls change rank, pivot, sample, sign, truncation, or selected constant branches, the check should report branch instability rather than treating the gradient as an HMC-ready object.

10 Source And Code Anchors

The source anchors used here are deliberately narrow.

- Zhao–Cui equations (1)–(3): state-space transition, observation, and parameter notation [1].
- Zhao–Cui equations (9)–(12) and Algorithm 1: recursive posterior update, nonseparable sequential density $q_t(x_t, \theta, x_{t-1})$, TT approximation, and normalizer construction [1].
- Zhao–Cui equation (13) and Lemma 1 vicinity: squared-TT defensive approximation $\hat{\pi} = \phi^2 + \tau\lambda$ and its normalizer [1].
- Zhao–Cui Proposition 2: marginal density construction from a normalized squared-TT approximation [1].
- Zhao–Cui Algorithm 2: sequential estimation using squared-TT approximations [1].
- Zhao–Cui Section 4.1: exact joint normalizer $z_t = \int \pi_t(x_t, \theta, x_{t-1} \mid y_{1:t}) dx_t d\theta dx_{t-1}$ is the evidence $p(y_{1:t})$, and the algorithm approximates q_t because the exact recursion is not directly evaluable [1].
- Zhao–Cui Proposition 4: conditional KR maps from the squared-TT marginal/conditional construction [1].
- Cui–Dolgov Sections 2.2–2.4 and Section 3: inverse Rosenblatt transport, functional TT representation, TT-based inverse Rosenblatt construction, and squared inverse Rosenblatt transport substrate used to interpret conditional KR maps [2].
- Companion code snapshot: `models/full_sol.m` updates the scalar by `log(sirt.z)-const`, and `deep-tensor.dev/src/@TTSIRT/marginalise.m` sets `obj.z = obj.fun.z + obj.tau`.

11 What Is Not Proved

This note does not prove posterior accuracy of the TT approximation, global differentiability of adaptive TT-cross/rank-changing code, HMC convergence, production readiness, GPU/XLA readiness, or a default-method recommendation. It proves two narrower statements: normalized approximate filtering for a declared fixed branch, and exact same-scalar differentiation for that declared branch.

References

- [1] Zhao, Y. and Cui, T. (2024). Tensor-train methods for sequential state and parameter learning in state-space models. *Journal of Machine Learning Research*, 25(244):1–51.
- [2] Cui, T. and Dolgov, S. (2022). Deep composition of tensor-trains using squared inverse Rosenblatt transports. *Foundations of Computational Mathematics*, 22:1863–1922. DOI: 10.1007/s10208-021-09537-5.